微算機實習**Lab02**

組別:第**14**組

資工三 **112590034** 高翎竣
資工三 **112590032** 陳俊彥
資工三 **112590453** 詹東儒

日期:**114**年**10**月**3**日

一、實驗內容

本次實驗中我們學到了如何透過GPIO與C/C++控制TX2上的LED燈，並了解到了linux作業系統file system的運作原理。

二、實驗過程及結果

I. 實驗過程:

我們按照簡報上所提供的程式碼，實作了針對gpio的function: `export`、`unexport`、`set direction`、`set value`的程式碼。我們另外也透過物件導向的方式，將這些function call封裝進物件，也將GPIO的pin腳封裝成更好理解的數字。

II. 實驗結果:

可以使用 `./LED {LED_ID} {on/off}` 來操作LED的開與關，和 `./LED blink {num}` 操作LED的閃爍。

III. 遇到的問題以及怎麼解決:

在撰寫程式碼的過程中，我們發現了兩個問題:

1. 每次開關LED都需要call好幾個function
2. 每次更改pin腳都需要額外去尋找對應的GPIO編號

針對第一點，我們實作了`class LED_CTRL`，目的是為了簡化LED開關的過程，在初始化該LED的物件後，只需要call `myLedObj->enable()` 即可打開該LED，call `myLedObj->disable()` 即可關閉該LED。

針對第二點，我們實作了 `struct LOOKUP::PIN`，其目的是快速尋找每個pin腳所對應的pgio，例如 `LOOKUP::PIN::P15` 就會回傳 `255`，`LOOKUP::PIN::P18` 就會回傳 `481`。

這個struct是我們這組設計的，因為他簡潔、使用方便，我們有將這段程式碼分享給同學使用。

三、程式碼

```cpp
/** USAGE:
*   LOOKUP::PIN::P15 -> 255
*   LOOKUP::PIN::P18 -> 481
*   etc etc etc
*/

namespace LOOKUP {
    struct PIN {
        enum : int {
            P7 = 396,
            P13 = 397,
            P15 = 255,
            P16 = 296,
            P18 = 481,
            P22 = 254,
            P29 = 398,
            P31 = 298,
            P32 = 297,
            P33 = 389,
            P37 = 388,
        };
    };
}
```

```cpp
#include "gpio_LED_CTRL.hpp"

namespace gpio {

LED_CTRL::LED_CTRL(int gpio) {
    gpio_id = gpio;
}

LED_CTRL::~LED_CTRL() {}

void LED_CTRL::enable() {
    util::gpio_export(gpio_id);
    util::gpio_set_dir(gpio_id, "out");
    util::gpio_set_value(gpio_id, 1);
}

void LED_CTRL::disable() {
    util::gpio_set_value(gpio_id, 0);
    util::gpio_unexport(gpio_id);
}

};
```

```cpp
int main(int argc, char* argv[]) {
    if (argc < 3) perror("insufficient parameters.");
    std::shared_ptr<gpio::LED_CTRL> LED1 = std::make_shared<gpio::LED_CTRL>(LOOKUP::PIN::P7 );
    std::shared_ptr<gpio::LED_CTRL> LED2 = std::make_shared<gpio::LED_CTRL>(LOOKUP::PIN::P13);
    std::shared_ptr<gpio::LED_CTRL> LED3 = std::make_shared<gpio::LED_CTRL>(LOOKUP::PIN::P15);
    std::shared_ptr<gpio::LED_CTRL> LED4 = std::make_shared<gpio::LED_CTRL>(LOOKUP::PIN::P32);

    std::string arg1 = std::string(argv[1]);
    std::string arg2 = std::string(argv[2]);

    if (arg1 == "LED1") {
        if (arg2 == "on") LED1->enable();
        else LED1->disable();
    }
    else if (arg1 == "LED2") {
        if (arg2 == "on") LED2->enable();
        else LED2->disable();
    }
    else if (arg1 == "LED3") {
        if (arg2 == "on") LED3->enable();
        else LED3->disable();
    }
    else if (arg1 == "LED4") {
        if (arg2 == "on") LED4->enable();
        else LED4->disable();
    }
    else if (arg1 == "blink") {
        blink(LED1, LED2, LED3, LED4, std::stoi(argv[2]));
    }

    return 0;
}
```

```
void blink(std::shared_ptr<gpio::LED_CTRL> LED1,
           std::shared_ptr<gpio::LED_CTRL> LED2,
           std::shared_ptr<gpio::LED_CTRL> LED3,
           std::shared_ptr<gpio::LED_CTRL> LED4, int count) {
    LED1->enable();
    LED2->enable();
    for (int i=0; i<count; ++i) {
        sleep(1);
        LED1->disable();
        LED2->disable();
        LED3->enable();
        LED4->enable();
        sleep(1);
        LED1->enable();
        LED2->enable();
        LED3->disable();
        LED4->disable();
    }
    sleep(1);
    LED1->disable();
    LED2->disable();
}
```

四、本次實驗過程說明與解決方法

在本次實驗中，我們依照簡報所提供的程式碼，實作了控制 GPIO 的基本功能，包括 **export**、**unexport**、**set direction**、**set value** 等操作，並採用物件導向設計，將相關 function call 封裝成物件，並以易於理解的 pin 腳代號進行對應。

實驗結果顯示，系統可透過指令
`./LED {LED_ID} {on/off}` 操作 LED 的開關，
以及 `./LED blink {num}` 控制 LED 閃爍。

在開發過程中，我們遇到兩項主要問題並進行改良：

1. **LED** 操作需多次呼叫 **function** → 透過實作 `LED_CTRL` 類別，簡化操作流程，只需呼叫 `enable()` 或 `disable()` 即可控制 LED。

2. 更改 **pin** 腳需手動查找 **GPIO** 編號 → 建立 `LOOKUP::PIN` 結構以快速對應，如 `LOOKUP::PIN::P15` 對應 255、`LOOKUP::PIN::P18` 對應 481，提升開發效率。

五、分工

學號、組員	貢獻比例	工作內容
112590034 高翎竣	40%	程式撰寫、報告撰寫
112590032 陳俊彥	30%	程式除錯、報告撰寫
112590453 詹東儒	30%	程式除錯、報告撰寫